PyMentor

A Personalized Multi-Agent Python Tutor | CSAI 422 Capstone, Option B

Seif Mohamed (202301506) | Patrick Saweris (202301486)

1. Problem and Objectives

Generic chatbots explain Python without knowing a learner's history, may give away homework answers, and can generate unsupported explanations. PyMentor is a grounded, curriculum-aware tutor that adapts to learner ability, remembers progress across sessions, records misconceptions, and applies hint-first teaching.

- Subject: introductory Python programming (CSAI 106 level).
- Providers: optional Groq API and local Ollama with qwen3:4b; final metrics used Ollama.
- Required orchestration framework: LangGraph.
- Core goal: improve learning while preserving groundedness and pedagogical discipline.

2. System Architecture

The application is a compiled LangGraph state machine. Shared state carries the student and session IDs, current message, routed intent, topic, profile, recent dialogue, retrieved contexts, confidence, guardrail flags, draft response, and final response.

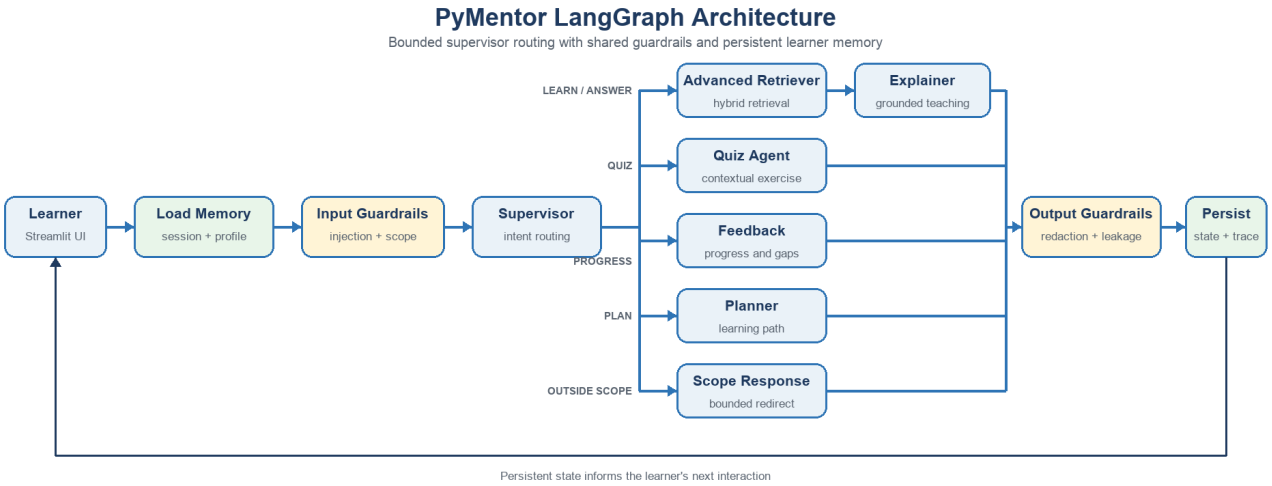

Figure 1. PyMentor architecture: bounded LangGraph routing with shared guardrails and persistent learner memory.

Node	Responsibility
Load Memory	Loads session history, profile, quiz history, and misconception log.
Input Guardrails	Detects injection, truncates oversized input, and marks answer requests.
Supervisor	Routes learning, quiz, progress, planning, and out-of-scope requests.
Retriever	Runs metadata-aware hybrid retrieval and reranking.
Explainer	Produces grounded, level-calibrated teaching with a check question.
Quiz Agent	Creates contextual exercises without revealing the solution.
Curriculum Planner	Recommends prerequisite-aware next learning steps.
Feedback Synthesizer	Summarizes evidence of mastery and remaining gaps.
Output Guardrails	Redacts secrets and blocks prompt leakage.
Persist	Stores the interaction for current and future sessions.

Routing is explicit and bounded. The model cannot execute arbitrary code or choose unregistered tools. This improves explainability, testability, and safety.

3. Advanced RAG Strategy

Course notes are divided by semantic headings into overlapping chunks of about 130 words with 25-word overlap. Every chunk stores title, section, topic, and difficulty. The baseline ranks raw query-token overlap. The final retriever combines BM25-style inverse-frequency weighting, topic and difficulty metadata boosts, title coverage, and deterministic reranking.

Pipeline	Context precision	Context recall
Naive lexical overlap	0.517	0.850
Hybrid + metadata + reranking	0.554	0.900

The final approach improved both measured metrics while remaining transparent enough to explain during the oral exam. Retrieved source IDs are shown in the UI. If no relevant context is available, the tutor states its limitation rather than inventing an answer.

4. Multi-Agent Design

The supervisor separates concerns across specialist nodes. The Explainer is optimized for instruction, the Quiz Agent for assessment, the Curriculum Planner for sequencing, and the Feedback Synthesizer for progress reporting. The retriever and guardrails are independent nodes so every specialist uses the same safety and evidence rules.

This design follows the course's supervisor pattern and avoids overlapping agent roles. A deterministic router handles obvious intents to reduce latency; generative models are used where language quality adds value.

5. Memory and Personalization

Memory layer	Stored information	Personalization use
Session	Ordered user and assistant messages	Coherent follow-ups without repetition
Student profile	Ability, goals, mastered and struggling topics	Level-calibrated explanations
Quiz history	Topic, score, details, timestamp	Evidence-based progress summaries
Misconception log	Specific error, topic, occurrence count	Targets recurring misunderstandings

SQLite was selected because it is persistent, inspectable, transactional, and adequate for the project scale. The same logical schema can move to PostgreSQL in production.

6. Guardrails

- Answer withholding: direct homework-solution requests trigger a hint, analogous example, and Socratic question.
- Scope enforcement: unrelated requests are routed to a bounded Python-curriculum response.
- Confidence calibration: weak retrieval causes an explicit limitation instead of hallucination.
- Prompt-injection defense: pattern checks, delimiters, fixed graph capabilities, and output leakage checks.
- Credential safety: API-key patterns are redacted, and secrets are loaded only from environment variables.

The controls use defense in depth. Deterministic checks run before and after generation, so compliance does not depend only on the model following a prompt.

7. Evaluation and Results

The reproducible suite contains 32 conversations spanning normal teaching, quizzes, three learner levels, direct-solution requests, injection attacks, scope violations, progress requests, and terse edge cases. Fourteen deterministic unit tests cover memory, retrieval, guardrails, learning assessment, and Qwen reasoning-output cleanup.

Metric	Result
Unit tests	14 passed
Retrieval precision improvement	0.517 to 0.554
Retrieval recall improvement	0.850 to 0.900
Deterministic pedagogical compliance	1.000
LLM-judge pedagogical compliance	1.000
Routing / grounded-response rate	1.000 / 1.000
RAGAS faithfulness	0.864 across 18 grounded cases
P95 / median latency	20.31 s / 15.13 s
Beginner pre/post quiz delta	+1.000
Intermediate pre/post quiz delta	+0.333
Advanced pre/post quiz delta	0.000 (already mastered)
Misconception detection accuracy	1.000 on controlled labeled set
Adversarial UI check	Prompt injection blocked and visibly flagged

The final run used local Ollama qwen3:4b on June 12, 2026. Case inspection exposed unfinished Qwen planning text in early quiz responses. We expanded routing/topic aliases, grounded quiz generation with retrieved sources, strengthened direct-solution detection, and rejected reasoning signatures so the graph falls back to deterministic grounded output. The corrected 32-case dataset contained no reasoning leaks.

8. Reflection and Future Work

The project demonstrates that measurable reliability comes from the application around the model: retrieval, explicit state, persistence, constrained routing, validation, and evaluation. The first retrieval experiment revealed taxonomy mismatches, leading us to normalize evaluation labels and document remaining difficult queries.

- Expand the corpus with complete CSAI 106 material and official Python documentation.
- Add embedding retrieval for paraphrases while retaining lexical and metadata signals.
- Implement structured quiz grading and automated misconception extraction.
- Add sandboxed Python execution for safe, testable coding exercises.
- Introduce human review for disputed grades and low-confidence explanations.
- Add authentication, encrypted storage, retention controls, and distributed tracing.

9. Repository and Submission Package

The public repository contains the complete runnable implementation and all evidence needed to inspect or reproduce the project:

- src/pymentor/: LangGraph workflow, specialist agents, hybrid retriever, memory, guardrails, and model providers.
- app.py: Streamlit live-demo interface.
- data/: source-tagged introductory Python knowledge base.
- evaluation/: 32-case evaluation suite, RAGAS and LLM-judge scripts, and measured result files.
- tests/: 14 deterministic regression tests.
- README.md and .env.example: installation, configuration, Ollama setup, and run instructions.
- docs/ and deliverables/: architecture, written report, tool disclosure, and supporting documentation.

Public GitHub repository: https://github.com/Tikaaaaaaa/pymentor-personalized-python-tutor